ACADEMIC PROJECT PRESENTATION REPORT

Decentralized Lottery Application (DApp)

Ethereum Smart Contract based Transparent, Autonomous aur Trustless Lottery System

STUDENT / DEVELOPER
Sahil Katkar

DOMAIN & TECH
Blockchain / Web3 / Solidity

TARGET ENVIRONMENT
Ethereum EVM (Ganache Local)

1. Project Overview & Problem Statement (Yeh Project Kyu Banaya?)

Traditional lottery systems (jaise government ya private lotteries) completely **centralized** hoti hain. Centralized system me common issues hote hain:

❌ Centralized Lottery Ki Problems

- **Lack of Transparency:** Players ko nahi pata hota ki kitna total paisa collect hua aur random number kaise generate hua.
- **Corruption Risk:** Organizer winner manipulate kar sakta hai ya funds lekar bhaag sakta hai.
- **Intermediary Fees:** Organizers huge cut/commission apne paas rakh lete hain.

✅ Decentralized Lottery (Hamara Solution)

- **100% Transparency:** Saara pool balance aur player list publicly blockchain ledger par visible hoti hai.
- **No Human Interference:** Winner selection logic autonomously Smart Contract execute karta hai.
- **Trustless & Secure:** Manager bhi contract ke prize pool se paisa apne account me nahi nikaal sakta. Winner ko direct auto-transfer hota hai.

2. System Architecture & Technology Stack

Yeh project ek full-fledged **Decentralized Web3 Application (DApp)** hai jisme 3 primary layers hain:

Layer	Technology	Role in Project
Smart Contract	Solidity (v0.4.17)	Lottery ke rules, participant registration, prize pool holding aur winner selection logic handle karta hai.
Blockchain Engine	Ganache CLI / RPC (EVM)	Local Ethereum node provide karta hai (Port `8545`, Chain ID `1337`) fast transaction mining ke saath.
Web3 Middleware	Web3.js (v1.0.0)	Frontend React UI aur Ethereum node ke beech JSON-RPC communication establish karta hai.

Layer	Technology	Role in Project
Crypto Wallet	MetaMask Extension	User identity authentication, private key management, aur transaction digital signature handle karta hai.
Frontend Interface	React.js (SPA)	Interactive dashboard provide karta hai jaha real-time balance, connected account, aur joined players display hote hain.

3. Smart Contract Working (Deep Dive into Lottery.sol)

Hamara smart contract `Lottery.sol` Ethereum blockchain par deployed hai. Iska code aur functions niche detail me samjhaye gaye hain:

```
pragma solidity ^0.4.17;

contract Lottery {
    address public manager;    // Contract deployer (Admin)
    address[] public players;  // Dynamic array: saare participants ke wallet addresses

    // Constructor: Deploy karte time deployer ka address save hota hai
    function Lottery() public {
        manager = msg.sender;
    }

    // 1. Enter function: Participant lottery me join karta hai
    function enter() public payable {
        require(msg.value > .01 ether); // Validation: Minimum fee > 0.01 ETH
        players.push(msg.sender);        // Player address array me add hota hai
    }

    // 2. Pseudo-random number generator
    function random() public view returns (uint) {
        return uint(keccak256(block.difficulty, now, players.length));
    }

    // 3. Winner selection & Prize distribution
    function pickWinner() public restricted {
        uint index = random() % players.length;  // Modulo operator se valid index milta hai
        players[index].transfer(this.balance);    // 100% prize pool winner ko send hota hai
        players = new address[](0);               // Next round ke liye array empty/reset
    }

    // Access Control Modifier: Sirf Manager ko pickWinner call karne deta hai
    modifier restricted() {
        require(msg.sender == manager);
        _;
    }

    // Current round ke saare players ka list return karta hai
    function getPlayers() public view returns (address[]) {
        return players;
    }
}
```

Key Solidity Concepts Explained:

- `payable` **Keyword:** Solidity me kisi bhi function ko Ether receive karne ke liye `payable` hona zaroori hota hai. Agar koi user `enter()` call karega to uska ETH contract account balance me jama ho jata hai.
- `modifier restricted()` **:** Yeh ek security guard ki tarah kaam karta hai. Yeh verify karta hai ki function call karne wala address `msg.sender == manager` hai ya nahi. Agar koi normal player `pickWinner()`

dabayega, to transaction turant **revert** ho jayegi aur gas refund ho jayegi.

- `this.balance` **:** Smart contract ka internal balance. Winner pick hote hi contract balance zero ho jata hai aur poora pool winner ke account me chala jata hai.
- **State Reset (** `players = new address[](0)` **):** Winner transfer hote hi dynamic array ko dynamically re-initialize kar diya jata hai taaki agla round fresh start ho sake.

4. Project Execution Flow (Kaise Run Hota Hai?)

> **Step-by-Step Flow:**
> **1. Ganache Node Start:** Terminal 1 me Ganache blockchain start hoti hai jisme 10 pre-funded accounts (1000 ETH each) create hote hain.
> **2. Contract Deployment:** Account 0 (`Sahil_katkar (Manager)`) se `Lottery.sol` deploy hota hai. Contract address `ui/src/contractConfig.js` me inject ho jata hai.
> **3. Players Joining:** Player (jaise `Ishwar_kumavat` ya `abdul_khan`) MetaMask connect karta hai aur 0.05 ya 0.1 ETH bhejkar join karta hai.
> **4. Winner Draw:** Manager MetaMask me account switch karta hai aur "Pick Winner" execute karta hai. Winner ko directly wallet me prize pool mil jata hai.

5. Terminal Commands (Project Run Karne Ke Steps)

Teacher ke saamne live demo dikhane ke liye sirf yeh 2 terminals use hote hain:

🖥️ Terminal 1: Local Blockchain Start Karna

```
npm run ganache
```

👉 Ganache RPC `http://127.0.0.1:8545` aur Chain ID `1337` par listen karna start kar deta hai. Is terminal ko open rakhna hai.

🖥️ Terminal 2: Deploy Contract, Seed Players & Start Frontend

```
# 1. Smart Contract ko deploy karna
npm run deploy:local

# 2. Test players add karna (Pool me ETH fund karna)
node scratch/add_players.js

# 3. React Frontend application launch karna
npm run ui
```

👉 Browser me `http://localhost:3000` automatically open ho jayega.

6. Teacher Ke Favorite Viva Questions & Smart Answers (Hinglish)

Jab aap teacher ko project explain karoge, wo ye key questions zaroor poochhenge:

Q1: Yeh `random()` function production me secure kyu nahi mana jata? (Very Important)

Answer: Sir, hamare code me `keccak256(block.difficulty, now, players.length)` use hua hai jo *pseudo-randomness* hai. Blockchain deterministic hota hai, aur `now` (timestamp) aur `block.difficulty` miners ko pehle se pata hote hain. Agar prize pool bohot bada ho, toh miner block timestamp ko manipulate karke apne favor me winner bana sakta hai. Isliye real production Ethereum mainnet par hum **Chainlink VRF (Verifiable Random Function)** use karte hain jo off-chain cryptographic proof provide karta hai.

Q2: Kya Contract Manager participants ka paisa chura sakta hai?

Answer: Nahi sir! Hamare contract me koi aisa function hi nahi hai jisme manager contract balance ko apne address par withdraw kar sake. `pickWinner()` function me strictly `players[index].transfer(this.balance)` likha hai, yaani paisa 100% kisi randomly selected player ko hi transfer hoga. Code is Law!

Q3: Frontend me Web3.js aur ABI ka kya role hai?

Answer: Smart contract EVM bytecode me compile hota hai. Frontend JavaScript ko contract ke saath interact karne ke liye **ABI (Application Binary Interface)** ki zaroorat hoti hai. ABI ek JSON schema hai jo

batata hai ki contract ke andar kaun se functions (jaise `enter`, `pickWinner`) aur variables hain. Web3.js is ABI aur contract address ko lekar MetaMask ke RPC provider ke through smart contract se baat karta hai.

Q4: Gas Fee kyu lagti hai aur read function me gas kyu nahi lagti?

Answer: `enter()` aur `pickWinner()` blockchain ki state change karte hain (array modify karte hain aur balance transfer karte hain), isliye EVM par computational effort aur storage ke badle miners ko gas deni padti hai. Jabki `getPlayers()` aur `manager` sirf `view` functions hain jo local node se state read karte hain bina kisi state modification ke, isliye read calls me 0 gas fee lagti hai.
